

pnpm commit

Script interativo de commit com validação de lint, typecheck e padrão de mensagem, com aplicação prática para a MyByte

1. Introdução

Mesmo em times pequenos e experientes, o momento do commit costuma ser o elo mais frágil do fluxo de desenvolvimento. É comum um desenvolvedor commitar às pressas, com uma mensagem genérica como "ajustes" ou "fix", esquecer de rodar o lint, ou só descobrir um erro de tipagem quando o pipeline do CI já falhou minutos depois, atrasando o review e o merge.

Este documento apresenta o conceito de "pnpm commit": um comando customizado que substitui o `git commit` tradicional por um fluxo guiado, interativo, que conduz o desenvolvedor passo a passo na hora de commitar e roda automaticamente as validações de qualidade antes de permitir que o commit seja criado. A ideia complementa diretamente o pipeline de validação de jira key apresentado anteriormente, adicionando uma camada de qualidade e padronização logo na origem, no computador do desenvolvedor.

2. Visão geral da solução

O "pnpm commit" nada mais é do que um script registrado no `package.json` do projeto, executado com `pnpm commit`. Em vez de o desenvolvedor digitar `git commit -m "..."` livremente, ele roda esse comando e responde a um pequeno questionário no terminal. Com base nas respostas, o script monta a mensagem de commit no formato padronizado da equipe, executa as verificações de qualidade configuradas e só então cria o commit de fato.

2.1 O que o fluxo guiado normalmente pergunta

- Tipo da alteração: feat, fix, refactor, docs, test, chore, perf, build ou ci
- Escopo da mudança, por exemplo checkout, auth ou api, para deixar claro qual parte do sistema foi tocada
- Chave da issue do Jira relacionada, reaproveitando a convenção MBX-123 já usada no pipeline de validação de commits
- Descrição curta e objetiva do que foi feito, em poucas palavras
- Se a mudança quebra alguma compatibilidade existente, com uma explicação adicional quando a resposta for sim

2.2 O que o script valida antes de liberar o commit

- Lint e formatação, rodando `eslint` e `prettier` apenas nos arquivos que serão commitados, via `lint-staged`
- Checagem de tipos do TypeScript, rodando `tsc --noEmit` no projeto
- Opcionalmente, testes unitários relacionados aos arquivos alterados
- Formato final da mensagem de commit, validando com `commitlint` antes de fechar o processo

Se qualquer uma dessas checagens falhar, o script interrompe o processo, mostra o erro de forma clara e nenhum commit é criado. O desenvolvedor corrige o problema localmente e roda `pnpm commit` novamente, sem nunca chegar a poluir o histórico do Git com um commit quebrado.

3. Ferramentas envolvidas

A solução combina algumas ferramentas conhecidas do ecossistema Node, todas leves o suficiente para rodar em segundos a cada commit.

inquirer	Biblioteca que monta o questionário interativo no terminal, usada dentro do script customizado de commit
husky	Gerencia os hooks do Git (pre-commit e commit-msg) de forma versionada junto do repositório
lint-staged	Roda lint e formatação apenas nos arquivos alterados, em vez do projeto inteiro, mantendo o processo rápido
commitlint	Valida se a mensagem final do commit segue o padrão definido pela equipe, funcionando como rede de segurança
typescript (tsc)	Executa a checagem de tipos do projeto antes de liberar o commit

4. Passo a passo de implementação

4.1 Instalar as dependências

Instalação das dependências de desenvolvimento

```
pnpm add -D husky lint-staged @commitlint/cli @commitlint/config-conventional inquirer chalk
```

4.2 Registrar o script no package.json

Trecho do package.json com os scripts necessários

```
{
  "scripts": {
    "commit": "node scripts/commit.mjs",
    "typecheck": "tsc --noEmit",
    "lint": "eslint .",
    "prepare": "husky"
  }
}
```

4.3 Criar o script interativo de commit

O coração da solução é um script Node que pergunta as informações do commit, roda as validações de qualidade e, se tudo passar, monta a mensagem final e executa o commit.

```
// scripts/commit.mjs
import { execSync } from "node:child_process";
import inquirer from "inquirer";
import chalk from "chalk";

const TIPOS = ["feat", "fix", "refactor", "docs", "test", "chore", "perf", "build", "ci"];
const JIRA_KEY_REGEX = /^[A-Z]{2,10}-[0-9]+$/;

function run(comando, descricao) {
  console.log(chalk.blue(`\n> ${descricao}...`));
  execSync(comando, { stdio: "inherit" });
}

async function main() {
  const respostas = await inquirer.prompt([
    { type: "list", name: "tipo", message: "Tipo da alteracao:", choices: TIPOS },
    { type: "input", name: "escopo", message: "Escopo (ex: checkout, auth):" },
    {
      type: "input",
      name: "jira",
      message: "Chave da issue do Jira (ex: MBX-123):",
      validate: (v) => JIRA_KEY_REGEX.test(v) || "Formato invalido, use algo como MBX-123",
    },
    { type: "input", name: "descricao", message: "Descricao curta da mudanca:" },
    { type: "confirm", name: "breaking", message: "Essa mudanca quebra compatibilidade?", default: false }
  ]);

  try {
    run("pnpm lint-staged", "Rodando lint e formatacao nos arquivos alterados");
    run("pnpm typecheck", "Verificando tipos com o TypeScript");
  } catch {
    console.log(chalk.red("\nCommit cancelado: corrija os erros acima e rode 'pnpm commit' de novo."));
    process.exit(1);
  }

  const prefixo = respostas.breaking ? `${respostas.tipo}!` : respostas.tipo;
  const mensagem =
    `${prefixo}(${respostas.escopo}): ${respostas.jira} ${respostas.descricao}`;

  execSync(`git commit -m "${mensagem}"`, { stdio: "inherit" });
  console.log(chalk.green(`\nCommit criado: ${mensagem}`));
}

main();
```

4.4 Configurar o lint-staged

O lint-staged garante que apenas os arquivos que fazem parte do commit sejam verificados, o que mantém o processo rápido mesmo em projetos grandes.

Configuração de lint-staged no package.json

```
{
  "lint-staged": {
    "*.ts,tsx,js,jsx": [
      "eslint --fix",
      "prettier --write"
    ]
  }
}
```

4.5 Configurar o commitlint como rede de segurança

O script interativo já monta a mensagem no formato correto, mas nada impede que alguém rode git commit diretamente, ignorando o fluxo guiado. Por isso, o hook de commit-msg do Husky continua validando o formato final da mensagem com commitlint, da mesma forma descrita no pipeline de validação de jira key.

commitlint.config.js validando o padrão conventional commits e a jira key

```
// commitlint.config.js
export default {
  extends: ["@commitlint/config-conventional"],
  rules: {
    "header-max-length": [2, "always", 100],
    "subject-case": [0],
  },
  plugins: [
    {
      rules: {
        "jira-key-presente": (parsed) => {
          const regex = /[A-Z]{2,10}-[0-9]+/;
          return [
            regex.test(parsed.raw),
            "A mensagem de commit precisa conter uma chave do Jira, ex: MBX-123",
          ];
        },
      },
    },
  ],
};
```

4.6 Registrar os hooks do Husky

Configuração dos hooks pre-commit e commit-msg com Husky

```
npx husky init
echo "pnpm lint-staged" > .husky/pre-commit
echo 'npx --no -- commitlint --edit "$1"' > .husky/commit-msg
```

5. Exemplo de uso no terminal

Na prática, o desenvolvedor deixa de digitar o commit manualmente e passa a rodar apenas o comando abaixo, respondendo ao questionário guiado.

Sessão de terminal usando pnpm commit

```
>> pnpm commit

? Tipo da alteracao: feat
? Escopo (ex: checkout, auth): checkout
? Chave da issue do Jira (ex: MBX-123): MBX-58
? Descricao curta da mudanca: adiciona validacao de cupom de desconto
? Essa mudanca quebra compatibilidade? Nao

> Rodando lint e formatacao nos arquivos alterados...
> Verificando tipos com o TypeScript...

Commit criado: feat(checkout): MBX-58 adiciona validacao de cupom de desconto
```

6. Casos práticos

CASO 1 | Bloqueio por erro de lint

Um desenvolvedor altera um arquivo e deixa uma variável não utilizada no meio do código. Ao rodar `pnpm commit`, a etapa de `lint-staged` encontra o problema, o ESLint aponta exatamente a linha e o motivo, e o processo é interrompido antes de qualquer commit ser criado. O erro é corrigido em segundos, sem nunca chegar ao pull request.

CASO 2 | Bloqueio por falha de typecheck

Uma alteração muda a assinatura de uma função, mas um outro arquivo que a utiliza não é atualizado. O `tsc --noEmit` acusa o erro de tipagem imediatamente durante o `pnpm commit`, evitando que o problema só seja descoberto minutos depois, quando o pipeline do GitHub Actions falhar no pull request.

CASO 3 | Padronização de mensagens para changelog automático

Como toda mensagem segue o mesmo formato, com tipo, escopo, chave do Jira e descrição, é possível gerar um changelog automatizado direto do histórico do Git, agrupando por tipo (novidades, correções, mudanças internas), sem depender de alguém escrever esse changelog manualmente a cada release.

CASO 4 | Onboarding de novo desenvolvedor ou freelancer

Um freelancer contratado para um projeto pontual nunca trabalhou com a convenção de commits da equipe. Ao rodar `pnpm commit` pela primeira vez, o próprio questionário ensina o padrão esperado, sem precisar de um documento extenso de onboarding ou de correções repetidas durante o code review.

7. Aplicação prática na MyByte

Para a MyByte, uma software house de porte pequeno com equipe enxuta, o maior custo de um processo de qualidade não é a ferramenta em si, é o tempo humano gasto revisando estilo de código, corrigindo erros de tipagem que o CI acusa depois, ou padronizando mensagens de commit manualmente durante o review. O pnpm commit desloca esse custo para o momento mais barato possível: antes do commit existir.

7.1 Ganhos específicos para uma equipe pequena

- Menos tempo de review gasto em comentários de estilo, já que lint e formatação já foram aplicados automaticamente antes do commit chegar ao pull request
- Menos ida e volta no CI, porque erros de tipagem e lint são pegos no computador do desenvolvedor, sem esperar o GitHub Actions rodar e falhar minutos depois
- Continuidade natural com o pipeline de jira key: o próprio questionário já pergunta a chave da issue, reduzindo a chance de alguém esquecer e o commit ser rejeitado mais adiante no pipeline
- Changelog e comunicação com o cliente ficam mais fáceis de montar, já que as mensagens seguem sempre o mesmo formato
- Onboarding mais rápido de freelancers e novos desenvolvedores, comum em software house que contrata por projeto ou sazonalidade

7.2 Proposta de adoção gradual

- **Passo 1:** instalar husky, lint-staged e commitlint no repositório piloto e configurar o pre-commit rodando apenas lint-staged
- **Passo 2:** criar o script scripts/commit.mjs com o questionário básico, reaproveitando o exemplo apresentado na seção 4.3
- **Passo 3:** adicionar a etapa de typecheck ao script, validando que o projeto builda sem erros de tipo antes do commit
- **Passo 4:** configurar o commit-msg do Husky com commitlint, reaproveitando a regra de jira key já usada no pipeline de validação de commits
- **Passo 5:** divulgar o comando pnpm commit para a equipe em uma reunião curta, deixando claro que é a forma padrão de commitar a partir daquele momento
- **Passo 6:** replicar a configuração para os demais repositórios ativos, priorizando os projetos com mais desenvolvedores trabalhando em paralelo

8. Boas práticas e cuidados

- Manter o typecheck rápido: em projetos grandes, considerar rodar a checagem completa de tipos apenas no pre-push ou no CI, e no pre-commit validar só os arquivos alterados, para não deixar o commit lento
- Nunca depender só do script interativo como camada de qualidade: os hooks do Husky (pre-commit e commit-msg) precisam continuar ativos, já que nada impede alguém de rodar git commit diretamente
- Definir uma via de escape documentada para emergências, como um commit de hotfix com `--no-verify`, mas com combinado claro de que isso deve ser exceção e não regra

- Manter o script simples: poucas perguntas, direto ao ponto, para não transformar o commit em um processo cansativo que a equipe tenta evitar
- Revisar periodicamente os tipos e escopos disponíveis no questionário conforme o projeto e a equipe crescem

9. Conclusão

O `pnpm commit` transforma o momento do commit, que hoje é livre e sujeito a esquecimentos, em um checkpoint de qualidade guiado e automático. Ele une num único passo o que normalmente estaria espalhado entre a memória do desenvolvedor, o code review e o pipeline do CI: lint, formatação, checagem de tipos e um padrão consistente de mensagem, já alinhado com a convenção de jira key usada pela equipe. Para uma equipe enxuta como a da MyByte, esse tipo de automação reduz atrito no dia a dia, evita retrabalho e mantém o histórico do Git limpo e útil, sem exigir esforço extra de gestão para isso acontecer.